

After the adjustments, `field.component` can function without the need for a conditional statement.

```
<template v-for="(field, key) in formFields">
  <Component
    v-model="formData[field.name]"
    :is="field.component"
    :type="field.type"
  />
</template>
```

Copyright Attribution: Medium

Field.component in forms.

Building forms in Vue.js using Mixins

While mixins support reusability, we want to delegate methods and computed properties to make managing the `<form-template>` variable easier. Mixins can eventually be mixed in with `<form-template>`.

We intended to save user feedback in `formData` in the Dynamic Component section.

We have helper variables in addition to `formData`:

- `formLength` — determines the length of the progress bar and navigation.
- `activeField` — just one field is shown on the screen.
- `IsComplete` — keeps note about whether or not a form has been completed.
- `isNext` — keeps track of shift path transitions (used later in Transition section)
- `isValid` — this property indicates whether or not an `activeField` is valid.

Provide and Inject

Provide and inject are marketed as a pair and must be used together. The `data()` method on a Vue instance is reactive by default. By default, data supplied with provide is not reactive. Any reactive data given as an entity, on the other hand, becomes measurable. The keyword here is object.

As a result, we store the above five variables in the `formState` object and then inject them into child components through dependency injection.

Declare a reactive data object